LEMBAR KERJA MAHASISWA (LKM) MINGGU 6

INTEGRASI QA & QC PADA MODEL SDLC

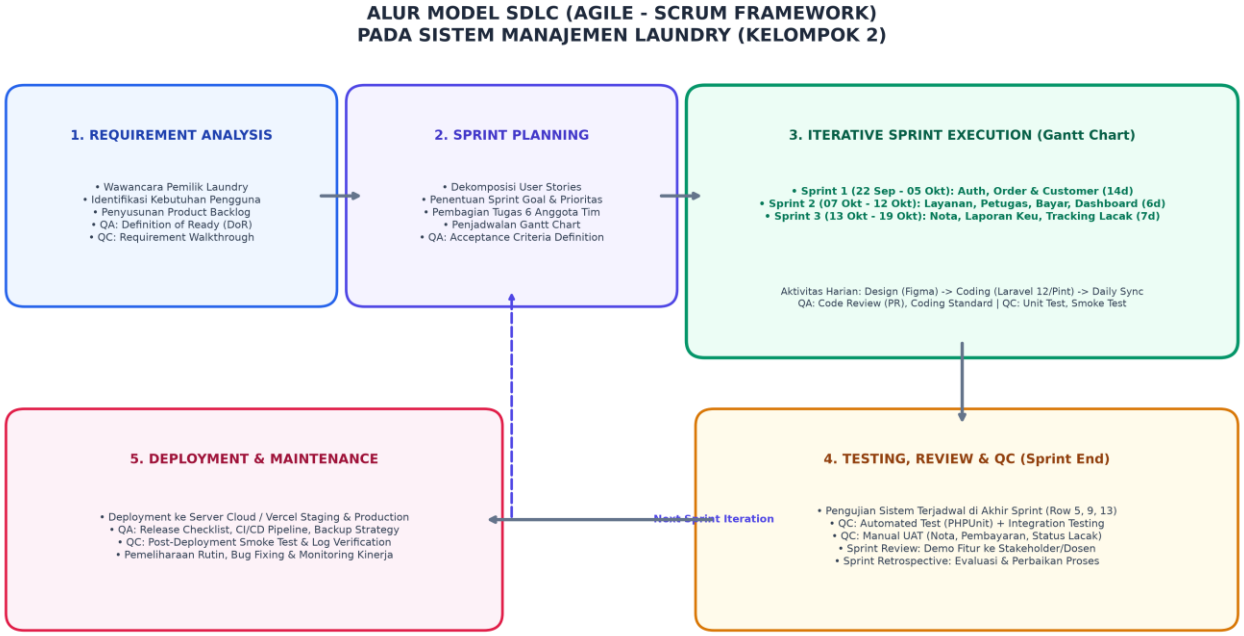
Mata Kuliah: Assurance Quality & Quality Control | Sub-CPMK: 5 & 6
Menyesuaikan Matakuliah Proyek Inovasi Digital (Dosen Pengampu: Bapak Rohim)

Kelompok	Kelompok 2
Anggota Kelompok	1. Adi 2. Alvin Dwi Lintang H. 3. Faqih 4. Yasir 5. Erna 6. Naila
Studi Kasus Proyek	Sistem Manajemen Laundry (Laundry Management System) Berbasis Web
Model SDLC Terpilih	Agile Software Development (Scrum Framework - 3 Iterative Sprints)
Tech Stack & Lingkungan	Laravel 12 (PHP 8.4), Blade, Tailwind CSS, MySQL, PHPUnit, Laravel Pint

BAGIAN A: PEMAHAMAN SDLC (Sub-CPMK 5)

1. Alur Model SDLC pada Proyek Sistem Manajemen Laundry

Untuk pengembangan Sistem Manajemen Laundry Kelompok 2, model SDLC yang diterapkan adalah **Agile Software Development dengan kerangka kerja Scrum (Iterative Sprints)**. Pilihan model ini tercermin secara eksplisit pada jadwal kerja proyek (Gantt Chart kelompok), di mana proses pengerjaan dibagi menjadi tiga siklus sprint iteratif dengan tonggak pengujian sistem di setiap akhir sprint, mulai dari analisis kebutuhan hingga pemeliharaan.



Gambar 1. Alur Integrasi SDLC Agile (Scrum) pada Sistem Manajemen Laundry Kelompok 2

Rincian tahapan alur SDLC proyek sesuai jadwal nyata Gantt Chart:

a. Requirement Analysis & Product Backlog: Tim melakukan wawancara dengan pemilik laundry untuk menggali kebutuhan operasional. Hasil analisis didekomposisi menjadi Product Backlog berisi user stories utama: autentikasi kasir/admin, pencatatan orderan kiloan/satuan, master jenis layanan, pencatatan transaksi pembayaran, cetak nota, laporan keuangan, dan pelacakan status cucian oleh konsumen.

b. Sprint Planning & Penjadwalan: Membagi backlog ke dalam 3 milestone Sprint dengan estimasi waktu dan alokasi kerja untuk 6 anggota (Adi, Alvin, Faqih, Yasir, Erna, Naila) secara hybrid (kolaborasi online via GitHub & tatap muka offline untuk sinkronisasi logika).

c. Iterasi Pengembangan (3 Sprints Berjalan):: • Sprint 1 (22 Sep - 05 Okt 2026, Durasi 14 Hari): Pembuatan modul Login & Logout multi-role serta pengelolaan data Orderan & Pelanggan. Diakhiri dengan Pengujian Sistem Sprint 1 pada 05 Oktober 2026.
• Sprint 2 (07 Okt - 12 Okt 2026, Durasi 6 Hari): Pembuatan fitur Mengelola Jenis Layanan & Petugas, Status Orderan & Sistem Pembayaran, serta Dashboard ringkasan bisnis. Diakhiri dengan Pengujian Sistem Sprint 2 pada 12 Oktober 2026.
• Sprint 3 (13 Okt - 19 Okt 2026, Durasi 7 Hari): Pembuatan fitur Cetak Nota kasir, Laporan Keuangan pendapatan, dan Halaman Pelacakan Status Cucian oleh Pelanggan secara mandiri. Diakhiri dengan Pengujian Sistem Komprehensif pada 16 - 19 Oktober 2026.

d. Testing, Sprint Review & Retrospective: Pada akhir tiap iterasi, dilakukan verifikasi kualitas internal (Quality Control) dan demonstrasi fungsional fitur. Feedback digunakan untuk menyempurnakan sprint berikutnya.

e. Deployment & Maintenance: Aplikasi di-deploy ke cloud environment (Vercel & Staging Server Docker) dengan database MySQL. Tahap pemeliharaan mencakup monitoring log error, perbaikan defect (bug fixes), serta optimasi query basis data secara berkala.

2. Alasan Kesesuaian Model Agile dan Satu Kelemahannya

Model Agile (Scrum) sangat sesuai untuk pengembangan Sistem Manajemen Laundry karena alasan-alasan berikut:

• **Dekomposisi Fitur Modular dan Inkremental:** Aplikasi laundry memiliki modul-modul bisnis yang saling independen dan dapat diselesaikan bertahap. Fitur inti kasir (order & pelanggan) dapat langsung difungsikan di Sprint 1, kemudian diperkaya dengan modul pembayaran & dashboard di Sprint 2, dan disempurnakan dengan reporting serta public tracking di Sprint 3.

• **Fleksibilitas Terhadap Penyesuaian Kebutuhan Pengguna:** Alur kerja operasional laundry di lapangan seringkali memerlukan penyesuaian (misal perubahan status pesanan: 'Baru -> Cuci -> Kering -> Setrika -> Siap Diambil', format nota thermal, atau validasi pembayaran). Agile memungkinkan tim mengintegrasikan perubahan tanpa merombak arsitektur sistem dari awal.

• **Manajemen Risiko dan Kualitas Terjadwal Dini:** Sebagaimana tertera pada Gantt Chart, tonggak pengujian sistem diadakan di akhir setiap sprint (Row 5, 9, dan 13). Ini mencegah terjadinya akumulasi cacat (defect accumulation) di akhir proyek.

• **Karakteristik Kolaborasi Tim Mahasiswa:** Dengan 6 orang pengembang dalam skema kerja hybrid (online/offline), rentang sprint 1-2 minggu memberikan ritme kerja (cadence) yang terukur, akuntabilitas personal yang jelas, dan evaluasi berkala.

Satu Kelemahan Utama Model Agile pada Proyek Ini:

Kelemahan paling nyata adalah kecenderungan dokumentasi teknis yang minim atau tertinggal (Lack of Comprehensive Documentation). Karena manifesto Agile memprioritaskan 'Working Software over Comprehensive

Documentation' dan tim didorong bergerak cepat memenuhi batas waktu sprint, dokumentasi arsitektur database, kamus data, spesifikasi API endpoint, dan changelog berisiko tidak terbarui dengan disiplin. Hal ini dapat menyulitkan proses maintenance atau serah terima sistem di masa mendatang.
Mitigasi Kelompok: Kelompok 2 menetapkan Quality Assurance berupa checklist 'Definition of Done (DoD)', di mana setiap fitur baru wajib menyertakan dokumentasi inline PHPDoc dan pembaruan migrasi database sebelum disetujui untuk di-merge.

3. Perbandingan Model Agile dengan Model Waterfall

Berikut adalah perbandingan mendalam antara model Agile (Scrum) yang diterapkan pada Sistem Laundry dengan model Waterfall konvensional berdasarkan karakteristik arsitektural dan keunggulannya:

Dimensi Perbandingan	Model Agile (Scrum) - Proyek Laundry	Model Waterfall (Konvensional)
Pendekatan & Siklus	Iteratif dan Inkremental; sistem dipecah menjadi sprint mingguan menghasilkan software fungsional bertahap.	Linier dan Sekuensial; fase berikutnya hanya dapat dimulai jika fase sebelumnya telah selesai 100%.
Manajemen Perubahan (Change Flexibility)	Sangat adaptif; perubahan kebutuhan dapat diakomodasi di setiap awal sprint planning berikutnya.	Kaku; perubahan requirement di tengah jalan memerlukan proses Change Request formal yang mahal.
Delivery of Value	Cepat dan berkelanjutan; kasir dapat mencoba modul order pada Sprint 1 (minggu ke-2).	Tertunda hingga akhir siklus; software baru dapat dioperasikan setelah seluruh fase coding selesai.
Keterlibatan Pengguna (Stakeholder)	Intensif di setiap demo sprint review untuk memberikan umpan balik langsung.	Hanya intensif di awal (analisis kebutuhan) dan di akhir (User Acceptance Test).
Integrasi Pengujian (QA & QC)	Pengujian berjalan kontinu di setiap sprint (Row 5, 9, 13 Gantt chart); cacat terdeteksi sangat dini.	Pengujian terisolasi hanya pada fase testing setelah seluruh coding selesai (risiko bottleneck tinggi).
Fokus Dokumentasi	Secukupnya (Just enough documentation); fokus utama pada kode yang berjalan dan terdokumentasi rapi.	Sangat ketat dan formal; dokumen tebal (SRS, SDD) menjadi prasyarat sebelum baris kode pertama ditulis.
Keunggulan Utama	Sangat unggul untuk aplikasi bisnis dinamis (laundry), meminimalkan risiko salah sasaran, dan feedback cepat.	Sangat unggul untuk sistem berskala besar dengan regulasi ketat, spesifikasi tetap, dan tim lintas vendor.

BAGIAN B: PERAN QA DAN QC (Sub-CPMK 6)

1. Contoh Aktivitas QA dan QC pada Sistem Manajemen Laundry

Berdasarkan arsitektur codebase Laravel 12 dan lingkungan kerja Kelompok 2, berikut adalah 2 contoh konkret aktivitas Quality Assurance (QA) dan 2 contoh konkret aktivitas Quality Control (QC):

- **Aktivitas QA 1: Standarisasi Coding Guidelines dan Automated Linter (Laravel Pint & PSR-12)**
  [Tujuan: Pencegahan Cacat (Defect Prevention) pada Tingkat Proses]
  Tim menetapkan standar penulisan kode seragam berbasis PSR-12 dan mengintegrasikan tool Laravel Pint pada seluruh environment pengembang (Adi, Alvin, Faqih, Yasir, Erna, Naila). Tool ini dikonfigurasi pada Git pre-commit hook untuk memformat kode secara otomatis sebelum di-push. Aktivitas ini mencegah inkonsistensi sintaks, typo variabel, arsitektur controller yang berantakan, serta memastikan standar kode Laravel yang seragam sehingga kode mudah dipelihara dan bebas dari defect akibat kesalahan format.

- **Aktivitas QA 2: Penerapan Peer Code Review dan Standar Definition of Done (DoD) pada Pull Request**
  [Tujuan: Penjaminan Kualitas Proses Kolaborasi Tim]
  Kelompok 2 menerapkan aturan wajib Code Review pada repositori GitHub. Setiap cabang fitur baru (misal fitur pembayaran atau tracking) tidak boleh digabung (merge) langsung ke branch 'main' tanpa disetujui (approved) minimal oleh 1 rekan tim. Reviewer mengecek apakah query database aman dari SQL Injection (menggunakan Eloquent/Query Builder terikat), apakah validasi Form Request telah lengkap, dan apakah otentikasi role middleware kasir vs admin sudah terpasang.
- **Aktivitas QC 1: Eksekusi Automated Feature & Unit Testing Menggunakan PHPUnit**
  [Tujuan: Deteksi Cacat (Defect Detection) pada Produk Perangkat Lunak]
  Menjalankan suite pengujian otomatis pada repository proyek, seperti 'tests/Feature/LaundryFlowTest.php' dan 'tests/Feature/AuthTest.php'. Tes ini mengeksekusi logika nyata aplikasi untuk mendeteksi apakah terjadi bug: memastikan kalkulasi tarif (berat cucian x harga layanan) menghasilkan angka yang tepat, status order berubah secara valid (Pending -> Diproses -> Selesai), dan pengguna tanpa role kasir ditolak saat mengakses endpoint transaksi.
- **Aktivitas QC 2: Manual User Acceptance Testing (UAT) dan Cross-Device UI Testing**
  [Tujuan: Validasi Cacat Produk Akhir terhadap Kebutuhan Pengguna]
  Melakukan pengujian langsung terhadap artefak produk yang sudah jadi: menguji pencetakan nota transaksi pada printer thermal kasir (memastikan teks nota tidak terpotong, rincian biaya presisi, dan barcode terbaca scanner), serta menguji antarmuka halaman Pelacakan Cucian (Tracking) pada berbagai resolusi smartphone untuk memastikan pelanggan dapat melihat progress laundry mereka secara akurat dan responsif.

2. Alasan QA Disebut Berorientasi Proses dan QC Berorientasi Produk

Perbedaan mendasar antara QA dan QC terletak pada objek sasaran, orientasi waktu, dan tujuan filosofisnya:

Mengapa QA Disebut Berorientasi Proses (Process-Oriented)?

QA berfokus pada METODE, ATURAN, PEDOMAN, dan ALUR KERJA yang digunakan dalam membangun perangkat lunak. QA bersifat proaktif (preventive/pencegahan), dengan asumsi bahwa jika proses produksinya dirancang, distandarisasi, dan diawasi dengan benar, maka produk yang dihasilkan secara otomatis akan memiliki kualitas tinggi dan bebas dari cacat. QA menjawab pertanyaan esensial: 'Are we building the product in the right way?' (Apakah kita membangun produk dengan cara yang benar?). Contoh QA bukan memeriksa baris kode yang error, melainkan menetapkan SOP code review, memilih standar penulisan kode, dan menyusun rancangan strategi pengujian sebelum pengembangan dimulai.

Mengapa QC Disebut Berorientasi Produk (Product-Oriented)?

QC berfokus pada ARTEFAK NYATA atau HASIL AKHIR yang diproduksi (source code, executable, antarmuka, API endpoint, nota cetak, database). QC bersifat reaktif (detective/pendeteksian), dengan tujuan mengeksekusi sistem untuk menemukan cacat (bug/defect) yang masih lolos sebelum produk tersebut diserahkan ke tangan pengguna akhir. QC menjawab pertanyaan esensial: 'Did we build the right product, and does it work correctly?' (Apakah kita telah menghasilkan produk yang benar dan berfungsi sesuai spesifikasi?). Contoh QC adalah menjalankan PHPUnit test runner, melakukan penetrasi celah keamanan form login, dan memverifikasi nota pembayaran.

Analogi Komparatif pada Bisnis Laundry:

• QA (Proses): Penyusunan Standar Operasional Prosedur (SOP) laundry oleh manajemen — takaran deterjen standar per 5 kg pakaian, suhu air maksimal 40°C, pemisahan pakaian putih dan pakaian mudah luntur sebelum masuk mesin, serta jadwal servis rutin mesin cuci setiap 2 minggu. Ini semua dirancang untuk mencegah pakaian rusak atau luntur sejak awal.

• QC (Produk): Pemeriksaan fisik pakaian oleh staf kasir/checker sebelum dibungkus plastik — menginspeksi kerah baju apakah masih ada noda tertinggal, memastikan pakaian sudah disetrika rapi tanpa lipatan kusut, mencocokkan jumlah potong baju dengan nota transaksi, dan mencium apakah pakaian sudah harum. Jika ditemukan noda, baju dikembalikan ke unit cuci untuk dibersihkan ulang.

BAGIAN C: INTEGRASI QA/QC PADA SETIAP TAHAP (Sub-CPMK 6)

1. Tabel Matriks Integrasi QA & QC pada Setiap Tahap SDLC

Tabel berikut merangkum pemetaan aktivitas penjaminan mutu (QA) dan pengendalian mutu (QC) beserta luaran/artefaknya pada setiap tahap siklus pengembangan Sistem Manajemen Laundry Kelompok 2:

Tahap SDLC	Aktivitas QA (Pencegahan Cacat)	Aktivitas QC (Pendeteksian Cacat)	Output / Artefak
Requirement Analysis	• Menyusun template user story & Acceptance Criteria terstandar. • Menetapkan Definition of Ready (DoR) sebelum item masuk sprint. • Workshop klarifikasi spesifikasi bisnis bersama stakeholder laundry.	• Requirement Walkthrough & Formal Inspection terhadap backlog. • Validasi kelengkapan skenario batas (edge-case) transaksi. • Analisis ambiguitas kebutuhan menggunakan checklist verifikasi.	• Dokumen SRS / Product Backlog terverifikasi. • Acceptance Criteria per fitur. • Matriks Ketertelusuran (Requirements Traceability Matrix - RTM).
Design	• Menetapkan UI/UX Design System (warna, tipografi Tailwind, ikon). • Merancang arsitektur database sesuai kaidah normalisasi (3NF). • Pedoman arsitektur keamanan: Role-Based Access Control (RBAC) & hashing.	• Design Review & Heuristic Evaluation prototipe Figma bersama tim. • Verifikasi integritas relasi foreign key pada ERD (orders, payments, services). • Validasi mockup alur transaksi kasir agar efisien dan bebas ambiguitas UI.	• Prototype UI Figma yang telah disetujui (Sign-off). • Skema ERD & Kamus Data ternormalisasi. • Dokumen Arsitektur Sistem (Class & Sequence Diagram).
Coding (Implementation)	• Menetapkan standar coding PSR-12 & konfigurasi Laravel Pint linter. • Penerapan branching strategy di GitHub (Feature Branching). • Kebijakan wajib Peer Code Review sebelum Pull Request di-merge.	• Static Code Analysis (PHPStan / SonarQube) mendeteksi syntax/dead code. • Pembuatan dan eksekusi Automated Unit Testing per method controller. • Validasi sanitasi input form (mencegah XSS dan SQL Injection).	• Source code bersih, lolos linter & terstandarisasi. • Pull Request ter-review dengan approval log. • Unit Test Suite & laporan code coverage awal.
Testing	• Penyusunan Test Plan & Test Strategy komprehensif. • Standarisasi template pembuatan Test Case (positif, negatif, batas). • Penyediaan environment Staging yang terisolasi dari database produksi.	• Eksekusi Automated Feature Testing (PHPUnit: alur order, nota, laporan). • Blackbox Functional Testing pada alur kasir, bayar, & tracking. • User Acceptance Testing (UAT) bersama calon pengguna laundry.	• Test Execution Report & Status Pengujian. • Bug / Defect Log pada Issue Tracker GitHub. • Berita Acara UAT & User Acceptance Sign-off.
Deployment & Maintenance	• Otomasi deployment pipeline (CI/CD) ke server staging/production. • Penyusunan Release Checklist & SOP migrasi database. • Prosedur automated backup basis data harian dan Disaster Recovery Plan.	• Smoke Testing & Sanity Testing langsung pasca-deployment rilis. • Verifikasi konfigurasi environment (.env) dan koneksi database produksi. • Monitoring log error berkala (Laravel Log Viewer) dan pelaporan bug.	• Live Production Application yang stabil. • Deployment Release Notes & Changelog. • Dashboard Monitoring & Laporan Maintenance Berkala.

2. Kesimpulan: Risiko Jika QA/QC Hanya Dilakukan di Akhir Fase Pengkodean

Melakukan aktivitas QA dan QC hanya pada akhir fase pengkodean (fase testing konvensional / Waterfall trap) merupakan anti-pattern kritis dalam rekayasa perangkat lunak modern. Berdasarkan analisis rekayasa mutu, terdapat lima dampak risiko fatal yang dapat terjadi:

- **Lonjakan Biaya Perbaikan Cacat Eksponensial (Cost of Defect Multiplier):**
  Berdasarkan kurva Barry Boehm, biaya memperbaiki bug yang terdeteksi di fase akhir atau produksi bisa membengkak 30 hingga 100 kali lebih mahal dibandingkan jika dicegah pada tahap requirement atau design. Misalnya, jika kesalahan desain relasi database transaksi laundry baru disadari saat sistem sudah jadi, tim harus

merombak skema basis data, mengubah puluhan migration file, merefaktor seluruh model Eloquent, dan menulis ulang tampilan blade terkait.

- **Cacat Struktural dan Arsitektural yang Sangat Sulit Diperbaiki:**
  Cacat yang berakar dari analisis kebutuhan atau desain arsitektur (seperti celah keamanan hak akses role atau ketidakmampuan menangani konkurensi nomor nota) bersifat sistemik. Memperbaikinya secara terburu-buru di akhir pengkodean seringkali hanya menghasilkan solusi tambal sulam (quick-and-dirty monkey patch) yang rapuh dan merusak integritas arsitektur sistem.
- **Terjadinya Testing Bottleneck dan Keterlambatan Rilis (Schedule Slipping):**
  Ketika ribuan baris kode dilemparkan ke tim pengujian sekaligus tanpa melalui continuous testing di tiap sprint, fase pengujian akan mengalami bottleneck yang parah. QA/QC akan kebanjiran ratusan temuan bug dalam waktu bersamaan, yang berujung pada penundaan jadwal rilis produk (schedule slipping) atau terpaksa merilis aplikasi dalam kondisi tidak stabil.
- **Penurunan Moral Tim Pengembang dan Technical Debt Masif:**
  Developer mengalami stres tinggi (burnout) karena harus melakukan bug fixing di bawah tekanan tenggat waktu yang mencekik. Perbaikan darurat di bawah tekanan tinggi hampir selalu memicu efek samping (regression bugs), di mana perbaikan satu modul justru merusak dua modul lain yang sebelumnya berfungsi normal.
- **Risiko Kegagalan Bisnis Fatal dan Kehilangan Reputasi Pengguna:**
  Bagi bisnis laundry, jika bug kritis lolos ke tahap operasional nyata — seperti salah kalkulasi total tagihan pelanggan, nota tidak dapat dicetak saat antrean panjang pelanggan kasir, atau hilangnya riwayat orderan cucian — dampaknya bukan sekadar masalah teknis, melainkan kerugian finansial langsung bagi pemilik usaha dan hilangnya kepercayaan pelanggan laundry secara permanen.

Rekomendasi Solusi Mutu: Penerapan Paradigma 'Shift-Left Testing'

Untuk memitigasi seluruh risiko di atas, Kelompok 2 secara konsisten menerapkan paradigma 'Shift-Left Testing', yaitu menggeser seluruh aktivitas penjaminan kualitas (QA) dan pengendalian kualitas (QC) ke tahap seawal mungkin dalam SDLC. Kualitas dibangun sejak perumusan User Stories dan perancangan skema sistem (QA proaktif), serta dikontrol secara kontinu melalui milestone pengujian sistem di akhir setiap sprint (Row 5, 9, dan 13 Gantt Chart). Dengan pendekatan ini, cacat dicegah sebelum muncul dalam kode, bug dideteksi dalam siklus hitungan hari bukan bulan, dan Sistem Manajemen Laundry dapat dirilis dengan jaminan keandalan, stabilitas, dan performa yang optimal.